

Deep Matching and Retrieval of Indic Script Documents

Pappu Meghana (S20230030400)

Artificial Intelligence and Data Science - Indian Institute of Information Technology, Sririty

Abstract

Matching and retrieval of Indic scripts such as Telugu is challenging due to their complex encoding representation, where a single symbol or akshara may consist of multiple Unicode symbols. For Roman scripts, traditional string matching algorithms like Levenshtein edit distance has been used. These edit distance based algorithms focus on syntactic similarity between two strings. As such, in this work, the deep learning model has been used to capture semantic similarity and learn the edit distance between Telugu words. The deep learning model takes source and destination words in Unicode format and learns the edit distance between them. This model has two parallel and similar networks called *Siamese Neural Network* that learn semantic features of source and destination words. These semantic features are used to learn the edit distance. After training the model, features are extracted and the edit distance is automatically computed by the model. Further, matching at Unicode level is not intuitive as there is a many-to-many mapping between aksharas and their Unicode representation. To handle this problem, matching is also performed at glyph level.

Keywords: Retrieval, Indic Scripts, Unicode Representation, Levenshtein edit distance, Siamese Network

1. Introduction

Retrieval of Indic script documents has become an essential task for enabling digital inclusion and advancing information accessibility in linguistically diverse regions. Given a query, finding the right document from the large collection of data requires accurate matching between a user query and documents in the database. In contrast to Roman scripts, matching Indic script documents is inherently more complex due to the unique nature of their fundamental structural unit, the *akshara*. Unlike a standard Roman character, an *akshara* is a composite unit consisting of a base consonant that may optionally be combined with consonant and vowel modifiers. This type of layering, where the components are appended vertically or horizontally to the core character, gives the script a hierarchical

and multi-dimensional structure.

A secondary challenge arises from the representation of Indic scripts. A significant gap exists between the internal machine representation, the Unicode encoding, and the visual representation of Indic scripts making the matching task non trivial. Indic Scripts are structurally complex compared to Roman Scripts. While Roman scripts typically exhibit a near one-to-one mapping between characters and Unicode representations, Indic scripts are compositional in nature, where the mapping between spoken forms and written representations is not always direct.

The internal machine representation of Indic scripts are defined by **UTF-8**, based on the Unicode encoding of linguistic units. Furthermore, there exists a many-to-many mapping between unicode sequences and *aksharas*. However, because these sequences are stored linearly, they often fail to reflect the true structural significance or the multi-dimensional nature of the *akshara*.

Email address: meghana.p23@iiits.in (Pappu Meghana (S20230030400))

Traditional matching algorithms, such as the *Levenshtein Edit Distance* (LED), rely on explicit alignment with quadratic complexity and treat symbols as atomic, isolated units. By failing to account for the non-linear structure of the *akshara*, they focus primarily on surface-level syntactic similarity rather than the underlying semantic similarity required for effective retrieval. This limitation often results in inaccurate results for Indic matching tasks, as the algorithms cannot perceive the structural relationship between encoded units. Therefore, a new approach must be modeled to capture the structural complexity of Indic scripts.

In this work, we propose a deep learning framework based on a *Siamese Network* architecture to learn a similarity function that respects these unique structural properties, using Unicode-level sequences to preserve script-level integrity. In this study, the proposed approach is evaluated using the Telugu script as a case study for Indic script matching and retrieval.

2. Problem Statement

The problem addressed in this work is matching and retrieval of semantically similar Telugu documents using deep learning algorithms. Given two Telugu words x_1 and x_2 , the objective is to learn a distance function, f , that represents their structural similarity.

$$y = f(x_1, x_2) \quad (1)$$

In this work, the Levenshtein Edit Distance (LED) is used as a distance function.

The deep learning model learns the distance between two Telugu words represented as a variable sequence of Unicones. Both the words are fed to the model in parallel to obtain the corresponding fixed size representations in the embedding space. These two representations are combined to learn the edit distance between them. For the purpose, Siamese Network architecture is used. During training, fixed size embeddings are learnt using supervised

learning based on the edit distance. These embedding representation are ultimately used to compute the edit distance between two Unicode words (query and database words) during inference time.

3. Scope of the Project

Matching and retrieval can be performed at different scripts (like Telugu, Kannada, Devanagari etc.), levels (like *akshara*, word, sentence, paragraph, page, document etc.) and different representations (like UTF-8 encoding, UNICODE, Glyph etc.). In the current work, the focus is on the matching and retrieval of Telugu UNICODE words. Further, it is assumed that the words are syntactically valid without errors.

4. Motivation

Unlike Roman scripts, the processing of Indic script documents is challenging due to their complex representation. In Roman scripts, there is a one-to-one correspondence between a symbol and its encoded representation. But Indic scripts such as Telugu, do not have this simple representation. The fundamental symbol unit in Telugu is the *Akshara*. Akshara is composed of different components such as base consonant, followed by optional consonant and vowel modifiers. Each component is called a **Glyph**. As such, each Akshara is represented as a sequence of one or more Unicode code points depending on its constituent components. Also, a single Unicode code point may represent different components in an Akshara. Further, Unicones are internally represented and stored using a UTF-8 encoding format in machines. Given the complexity in these visual glyph, Unicode and internal machine representations, there are challenges in the processing of Indic scripts.

5. Literature Review

String similarity and approximate matching have traditionally been studied using algorithmic approaches such as Levenshtein Edit Distance and dynamic programming based alignment methods. Foundational work in approximate string matching highlights the effectiveness of edit-distance based techniques for structured sequence comparison, while also noting their quadratic computational complexity and lack of adaptability to data-driven variations [5]. These limitations become more pronounced when dealing with scripts that exhibit variable-length encoding and compositional structure.

Recent research has explored learning-based alternatives where similarity between symbolic sequences is modeled using neural networks instead of predefined alignment rules. Metric learning frameworks aim to learn embedding spaces in which distances between representations reflect semantic or structural similarity [1]. Such approaches are particularly suitable for structured data where hand-crafted distance functions may fail to capture complex relationships.

Siamese neural network architectures provide a principled framework for similarity learning by employing shared encoders to map paired inputs into a common embedding space, followed by a distance or comparison function [2]. Instead of performing explicit alignment during inference, these models implicitly learn transformation-invariant representations of sequences, making them effective for tasks involving noisy inputs, spelling variations, and structural differences.

For sequential symbolic data, recurrent neural networks have been widely used to model variable-length sequences and capture long-range dependencies across tokens [3]. This is particularly relevant for Unicode-based representations, where a single linguistic unit may correspond to multiple encoded symbols, requiring context-aware sequence modeling rather than isolated character comparison.

In the context of Indic scripts, prior studies emphasize

that direct application of character-level methods designed for alphabetic languages is insufficient due to the many-to-many relationship between visual glyphs, linguistic units (aksharas), and Unicode encodings [4]. Indic writing systems exhibit compositional structures, conjunct formations, and encoding variability that complicate traditional string similarity measures.

Additionally, neural sequence models have shown sensitivity to structural noise and symbolic perturbations, indicating the need for representation-aware similarity learning rather than purely surface-level comparison [6]. This motivates the use of Unicode-level sequence encoders combined with Siamese distance learning to approximate edit-distance relationships in structurally complex scripts such as Telugu.

Overall, existing literature supports the transition from explicit alignment-based similarity computation to learned embedding-based distance modeling, especially for variable-length symbolic sequences and structurally rich writing systems. However, limited work directly addresses similarity learning for Indic scripts at the Unicode sequence level, which forms the primary focus of this study.

6. Input Representation

Linguistic length and encoded length are not equivalent in Indic scripts. Consider the following two Telugu words::

$$\text{లక్ష్మి} = \text{ల} + \text{క} + \text{్} + \text{ష} + \text{్} + \text{మ} + \text{ి} \quad (2)$$

$$\text{లత} = \text{ల} + \text{త} \quad (3)$$

Although both words appear short in their visual form, their Unicode representations differ substantially in length. This disparity is a direct consequence of the conjunct consonant structures present in Telugu, where the halant (U+0C4D, also referred to as Virama) serves as a combining mark that suppresses the inherent vowel of a consonant and joins it to the following character.

A critical design choice in this work is the representation of textual input. Telugu text is typically stored using

UTF-8 encoding, where each visible character may be represented using a variable number of bytes. Analysis of raw hexadecimal representations of Telugu text shows that visually similar strings can differ significantly at the byte level due to spacing, composition, or the presence of combining characters. As a result, modeling text as a sequence of bytes leads to sequences dominated by repeated prefix bytes, which do not correspond to meaningful linguistic units and introduce noise into sequence modeling.

To avoid this issue, each string x is represented as a sequence of Unicode code points:

$$x = (u_1, u_2, \dots, u_T), \quad u_i \in \mathcal{U} \quad (4)$$

where $\mathcal{U}$ denotes the Unicode vocabulary. Here, each u_i is represented as a one-hot encoded vector of size (128×1) , corresponding to the 128 possible code points in the Telugu Unicode block (0x0C00–0x0C7F). This representation preserves character-level structure while avoiding fragmentation caused by byte-level encoding. Since strings have variable lengths, sequences are padded to a fixed maximum length $T_{\max}$ for batch processing (if required).

The application of traditional string similarity measures to such sequences is limited in two respects. First, methods such as the Levenshtein Edit Distance (LED) treat each code point as an atomic unit and perform explicit character-level alignment, which is structurally unintuitive for scripts where a single linguistic unit spans multiple code points. Such measures capture only syntactic similarity at the encoding level and are insensitive to the compositional structure of aksharas. Second, the dynamic programming formulation of LED incurs a quadratic time complexity:

$$\mathcal{O}(|x_1| \cdot |x_2|) \quad (5)$$

which becomes computationally expensive as encoded sequence lengths grow due to multi-component akshara structures.

Tasks such as document retrieval, plagiarism detection, and OCR post-correction in Indic scripts require matching frameworks that go beyond syntactic alignment. A model capable of learning structural similarity directly from Unicode sequences — using LED as a supervision signal rather than as a direct computation — offers a principled alternative. The Siamese Network architecture is particularly suited to this requirement, as it learns a shared embedding space in which structurally similar sequences are mapped to proximate representations. This reduces inference-time complexity from $\mathcal{O}(|x_1| \cdot |x_2|)$ to $\mathcal{O}(|x_1|) + \mathcal{O}(|x_2|)$ and enables generalisation to unseen word pairs without explicit alignment.

Furthermore, the design of such an architecture for Indic scripts involves non-trivial choices regarding input representation, embedding strategy, and encoder architecture. Each of these choices has a direct bearing on how well the model captures the structural properties of Telugu Unicode sequences, motivating systematic investigation of the design space.

Now see the logical and intuitive flow of thoughts and presentation from sections 3 to 7. Also, go through the corrections in the text.

7. Design of DL Architecture

The DL architecture takes two input Telugu words in the UNICODE format and computes the LED between them. During training, the network learns the distance. After training, given a pair of two new input words, it computes the LED between them. The architecture has the following stages:

1. Inputs: UNICODE sequences of two words.
2. For each input word, UNICODE sequence of symbols are converted to the integer-encoding form. For the purpose, an alphabet of UNICODES is generated and stored in the form of a dictionary. All the UNICODES in a given word are converted to the corresponding

indices (positions) in the dictionary. For example, if the first UNICODE in the dictionary is 0C05, it is mapped to 1 (its index in the dictionary).

3. An Embedding layer projects all the integer-encoded symbols to a d -dimensional embedding space, where d is a hyperparameter.
4. Embedded vectors of all the integer-encoded symbols are stacked in the form a matrix and fed to the 1D-CNN layer to learn and extract spatial contextual features.
5. The spatial feature vectors of the two words are combined using any operation (addition, subtraction, concatenation etc.) and fed a fully-connected layer with one neuron.
6. The single neuron in the final layer predicts the LED between the two input words. During training, the mean squared distance between the predicted and the actual ground truth LED values is minimized over a batch of pairs of training samples. The weights of the network are accordingly updated by the MSE (mean-squared error) optimization.
7. During testing on new input words, the learnt weights are used to extract semantic features and LED is computed between them.

The above architecture is shown in Fig. 1 and discussed in detail below.

Replace the word "Siamese Network" with "Shared Network" in the block diagram.

7.1. Sequence Encoding

Each input vector of Unicode sequence, u_i , is mapped to a dense vector using an embedding matrix:

$$e_i = E[u_i], \quad E \in \mathbb{R}^{|\mathcal{U}| \times d} \quad (6)$$

where d is the embedding dimension.

The embedded sequence $(e_1, e_2, \dots, e_T)$ is then processed by a sequence encoder $\phi(\cdot)$, which maps the variable-length

input into a fixed-dimensional representation:

$$z_1 = \phi(x_1), \quad z_2 = \phi(x_2), \quad z \in \mathbb{R}^{d_{dim}} \quad (7)$$

The resulting representations are then combined using a comparison function $\psi(\cdot)$ to produce a predicted similarity or distance score:

$$\hat{y} = \psi(z_1, z_2) \quad (8)$$

The function $\psi(\cdot)$ is kept generic and may involve operations such as vector concatenation $(z_1; z_2)$, element-wise interactions, or inner products, followed by a learnable mapping. This allows the model to learn an appropriate notion of similarity directly from data, rather than relying on a fixed distance metric. This formulation avoids explicit character-level alignment during inference and instead learns a comparison function in the representation space.

7.2. Training

The objective of this work is to learn a distance function between pairs of strings in Indic scripts that correlates with classical edit-distance-based similarity, while remaining robust to script-specific encoding and composition effects. Given two input strings x_1 and x_2 , the goal is to predict a distance or similarity score $\hat{y} = f(x_1, x_2)$ that approximates a reference distance y , such as Levenshtein Edit Distance.

The model is trained using supervised pairs (x_1, x_2, y) , where $y = f(x_1, x_2)$ is a ground truth Levenshtein Edit Distance. A regression loss is used to align the predicted distance $\hat{y}$ with the ground-truth value:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2 \quad (9)$$

This objective encourages the embedding space to preserve edit-distance relationships between strings.

7.3. Design considerations for the shared network

Byte-level modeling was avoided because it fails to preserve the structural properties of UTF-8 encoded Indic text. Akshara-level modeling, although linguistically

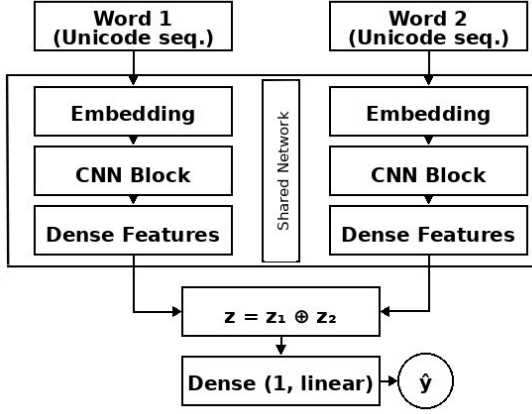


Figure 1: A Siamese Recurrent Architecture for Unicode-Level String Distance Learning

meaningful, introduces variable-length composite units that are difficult to integrate directly into standard recurrent architectures. Representing text as sequences of Unicode code points provides a balanced solution that is structurally stable, compatible with sequence models, and expressive enough to capture script-level variations.

(i) Embedding → CNN Pipeline: The RNN-derived embeddings are passed through a 1D Convolutional Neural Network to capture local structural patterns. The CNN acts as a structural feature extractor over short Unicode windows before global aggregation.

(ii) Embedding → RNN Pipeline: The embedded Unicode sequence is directly processed by a recurrent encoder (RNN/LSTM/GRU) to model long-range dependencies across code points. This captures sequential composition effects present in Indic scripts.

These two encoder pathways allow the model to either emphasize global sequential dependencies (RNN–RNN) or local compositional structure (RNN–CNN), which is particularly important for Telugu where the written glyph clusters and stored Unicode sequences are not strictly aligned.

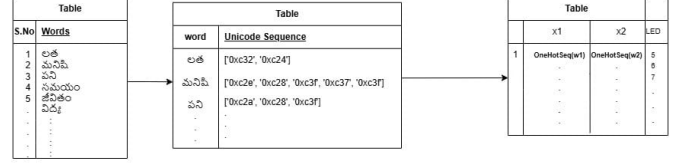


Figure 2: Preprocessing of the data: Unicode-aware preprocessing pipeline for Telugu words, definition of a fixed Unicode vocabulary (0x0C00–0x0C7F), conversion of variable-length strings into Unicode sequences, construction of pairwise string datasets with edit-distance supervision.

8. Present Work

For the current work, a vocabulary of 500 clean Telugu words is collected. Each word is converted into its corresponding Unicode code-point sequence, forming the input representation described earlier. A Unicode lookup table is maintained to map each word to its sequence of code points.

From this vocabulary, unordered word pairs are generated. This yields $\binom{500}{2}$ possible pairs. From these, a subset of 500 word pairs is sampled to construct the dataset.

For each selected pair (x_1, x_2) , the ground-truth distance y is computed using Levenshtein Edit Distance. Each data point in the dataset therefore has the form:

$$(x_1, x_2, y) \quad (10)$$

where x_1 and x_2 are Unicode code-point sequences corresponding to two words, and y is their edit distance.

This dataset is used to train the Siamese model to learn a mapping $f(x_1, x_2) \rightarrow y$.

9. Future Work and Timeline

Phase 2:

- Learning of embedding space for unicodes of dim < 128
- Replacing one-hot Unicode inputs with learned embedding representations

- Design of Siamese architecture with shared parameters for string encoding

Phase 3:

- Implement model, train test performance analysis
- Exploration of alternative comparison strategies between encoded strings (concatenation, dot product, cosine similarity, or learned distance functions)
- Train the Siamese network on the constructed Unicode pair dataset
- Evaluate alignment between predicted similarity and classical edit distance
- Perform qualitative analysis of learned representations

Phase 4: Generalization Glyph level

- The dataset can be expanded to include more words or larger vocabularies
- The vocabulary can be extended to explicitly include consonant and vowel modifiers, enabling the model to distinguish between base characters and their modified components during edit-distance-based supervision.
- The approach can be extended to other Indic scripts

Verification Using a Siamese Time Delay Neural Network,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 7, no. 4, pp. 669–688, 1993.

- [3] A. Graves, “Supervised Sequence Labelling with Recurrent Neural Networks,” *Springer*, 2009.
- [4] P. B. Krishnan, R. Jawahar, and C. V. Jawahar, “Script Independent Word Spotting in Indic Documents,” *Pattern Recognition*, vol. 48, no. 2, pp. 518–531, 2015.
- [5] G. Navarro, “A Guided Tour to Approximate String Matching,” *ACM Computing Surveys*, vol. 33, no. 1, pp. 31–88, 2001.
- [6] J. R. Belinkov and Y. Bisk, “Synthetic and Natural Noise Both Break Neural Machine Translation,” *International Conference on Learning Representations (ICLR)*, 2018.

References

- [1] A. Bellet, A. Habrard, and M. Sebban, “A Survey on Metric Learning for Feature Vectors and Structured Data,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 11, pp. 2172–2189, 2015.
- [2] J. Bromley, J. W. Bentz, L. Bottou, I. Guyon, Y. LeCun, C. Moore, E. Säckinger, and R. Shah, “Signature